

人力管理系统设计文档

Jesse Chan <cjx61234@gmail.com>

2026年3月31日 14:08

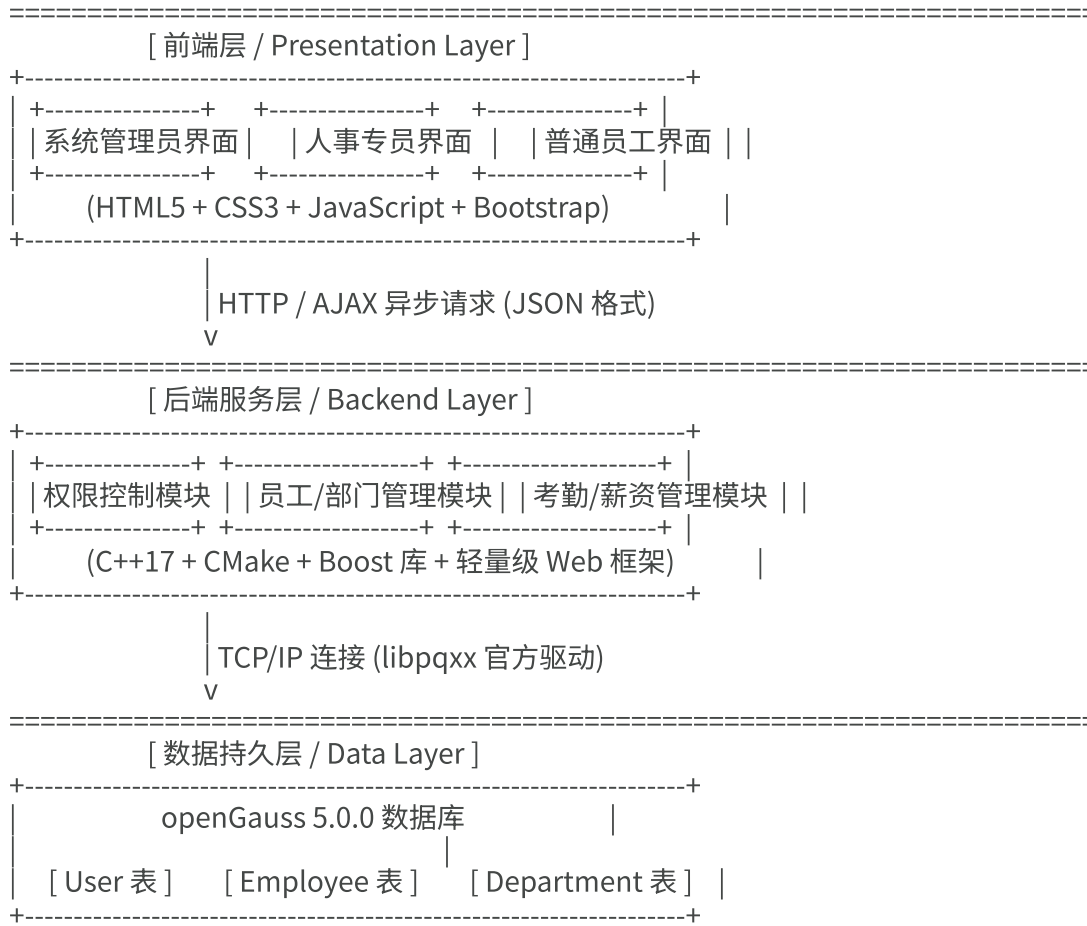
草稿

《企业人力管理系统 (HRMS) 全流程设计与开发文档》

1. 系统架构设计 (System Architecture)

系统严格采用三层架构，实现表示层、业务逻辑层与数据持久层的完全解耦。此设计确保了前后端分离，并支持在 Ubuntu 环境下开发后向 Windows 平台的平滑移植。

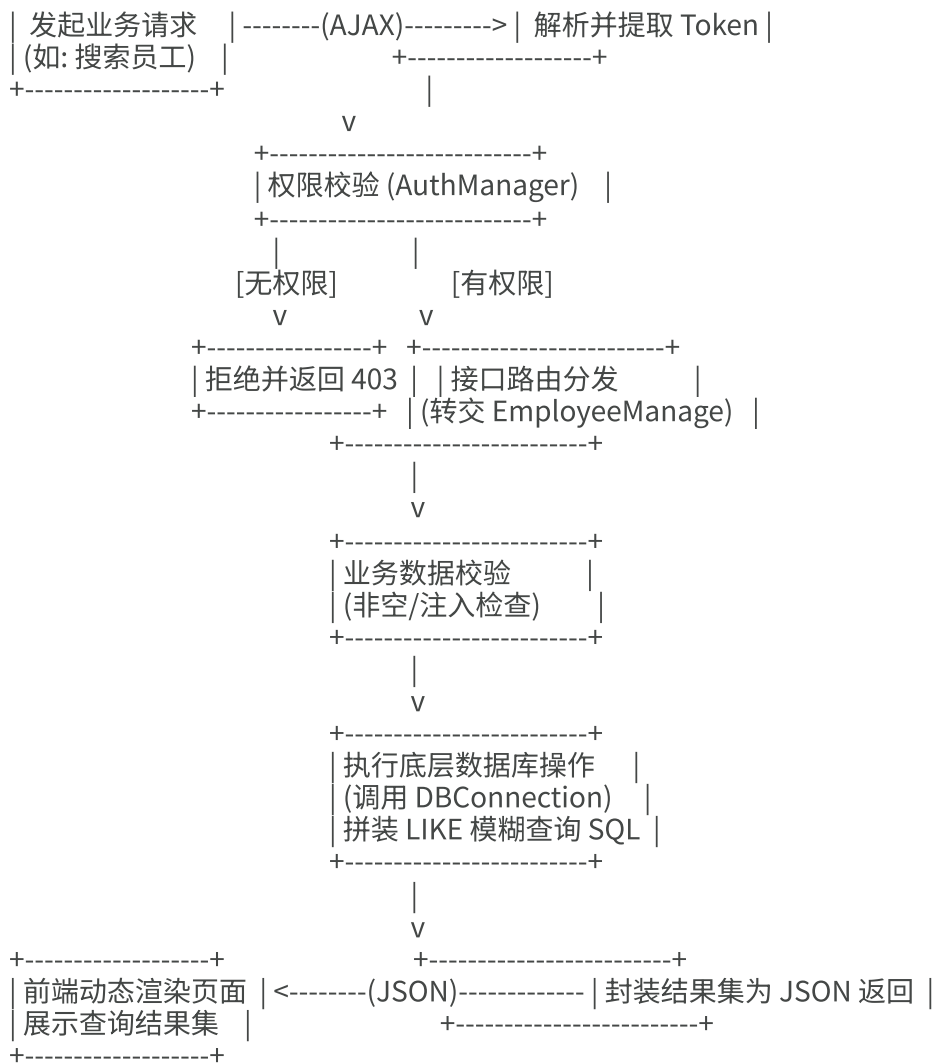
Plaintext



2. 核心业务逻辑 (Business Logic)

Plaintext

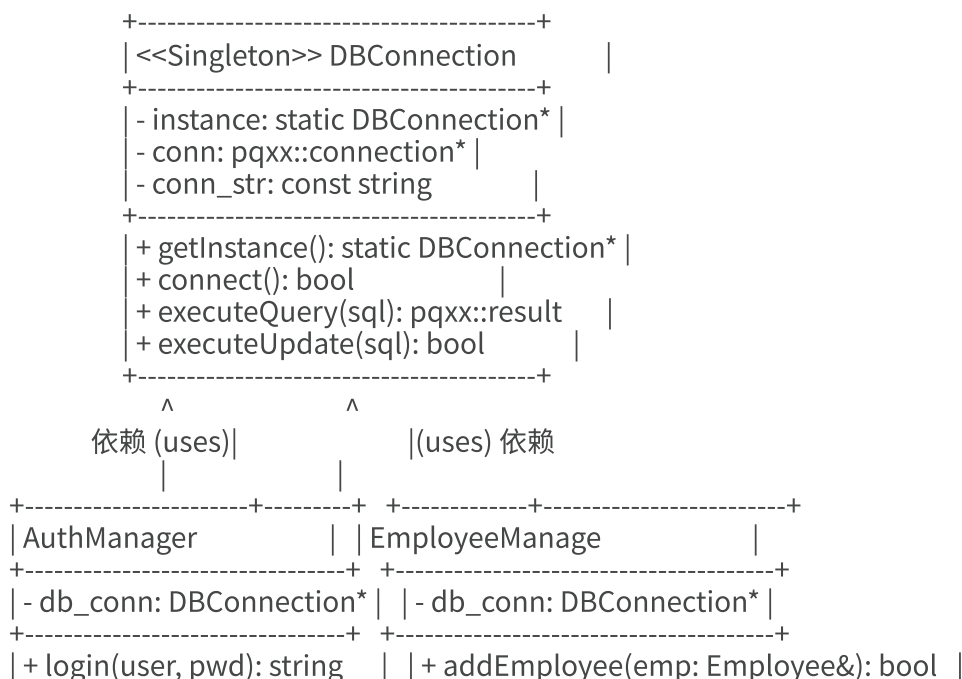


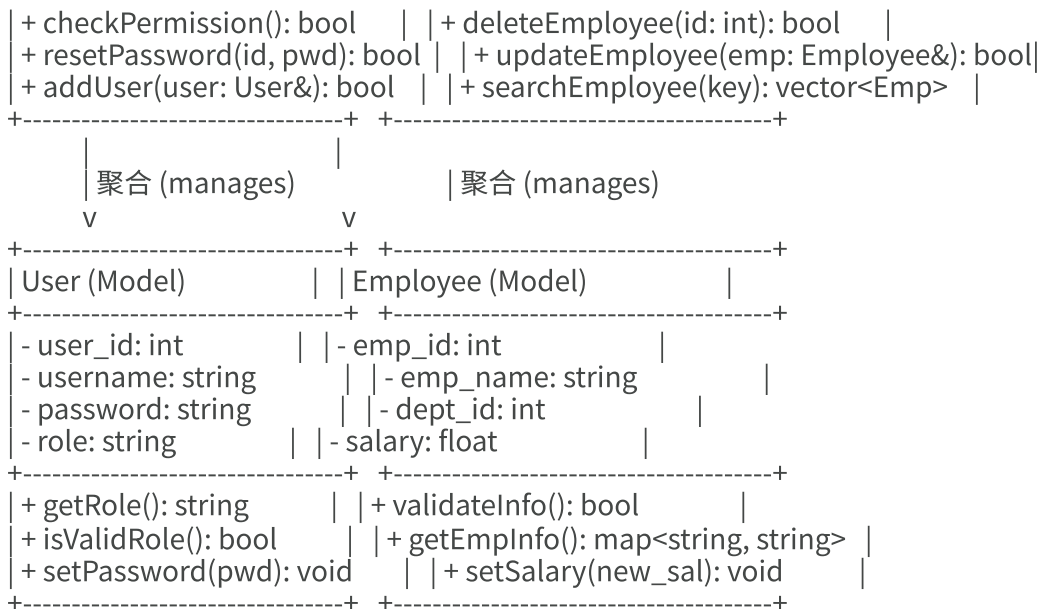


3. 系统核心类图 (Class Diagram)

类图展示了 C++ 后端工程的核心模块依赖。底层统一通过 DBConnection 单例管理数据库通信，上层分为鉴权模块与各类实体管理模块。

Plaintext





4. 团队分工与工作内容 (Division of Labor & Tasks)

组员A: 后端开发

组员B: 数据库开发和设计

组员C: 前端开发

组员D: 接口定义与测试

阶段一：契约制定与系统设计

在这个阶段，D 是系统分析师，核心需求是消除 A、B、C 之间的信息差，确立开发标准。

D 的工作流程：

1. **解析业务流**：根据业务逻辑图，梳理出所有需要的操作（如：登录、新增员工、模糊搜索、越权拦截）。
2. **定义数据结构 (给 A 和 B)**：编写 CoreModels.h，定义 User、Employee、Department 实体类的 C++ struct 或 class，确保 A 传给 B 的数据类型绝对一致。
3. **定义类接口 (给 A 和 B)**：根据类图，写出 EmployeeManage.h 和 AuthManager.h 的纯虚函数接口（如 virtual vector<Employee> searchEmployee(string keyword) = 0;）。
4. **定义网络 API (给 A 和 C)**：编写 Markdown 格式的 API 文档，规定前端 AJAX 请求的 URL、HTTP 方法（GET/POST）以及 JSON 的收发格式。
5. **召开评审会**：A、B、C 确认接口无误后，D 锁死接口文档。后续任何修改必须由 D 统筹。

阶段二：核心模块并行开发

接口定死后，大家就可以在自己的领域里“疯狂输出”了。

组员 A：系统架构与后端逻辑 (Backend & Architecture)

- **需解决的需求**：搭建稳定的 C++ 服务端，实现基于角色的权限控制（RBAC），并承接前端请求。
- **工作流程**：
 1. **环境初始化**：在 Ubuntu 中配置好 GCC 9.0+ 和 CMake 3.15+。在搭建开发环境和后续编译运行 hrms 后端服务时，直接使用标准用户权限进行操作即可，无需切换到 root 权限。这不仅能避免生成的文件被锁死，还能在最大程度上保证 Linux 宿主机的系统安全。
 2. **引入网络库**：配置 CMake，引入 Boost 或轻量级 HTTP 库，搭建一个能监听本地端口的 C++ Web Server。
 3. **实现 AuthManager**：开发登录校验逻辑，成功后生成 Token（如 JWT 或简单的哈希字符串），并编写拦截器逻辑：校验失败直接返回 {"error": "无权限"}。

4. **业务路由分发**: 接收前端 JSON, 解析后调用 B 提供的 `EmployeeManage` 接口, 再将 B 返回的 C++ 对象组装成 JSON 返回给前端。

组员 B: 数据持久化与底层封装 (Database & DAO)

- **需解决的需求**: 设计 openGauss 表结构, 实现安全、高效的数据库连接, 完成增删改查及模糊匹配 SQL。
- **工作流程**:
 1. **物理建表**: 编写 .sql 脚本, 在 openGauss 中创建三张核心表, 严格设置主外键约束 (如 `Employee.dept_id` 关联 `Department`)。
 2. **单例连接池**: 实现 `DBConnection.cpp`, 封装 `libpqxx` 的 `pqxx::connection`, 确保全局只有一个数据库连接实例, 避免连接耗尽。
 3. **实现 CRUD 接口**: 继承 D 定义的接口, 编写 `EmployeeManage.cpp`。将 A 传来的 `Employee` 对象转化为 `INSERT/UPDATE` 语句执行。
 4. **攻坚模糊查询**: 重点实现 `searchEmployee`, 使用 `pqxx` 的参数化查询 (防止 SQL 注入), 拼装 `LIKE '%keyword%'` 语句, 并将 `pqxx::result` 解析打包成 `vector<Employee>` 返回给 A。

组员 C: 前端交互与界面渲染 (Frontend & UI)

- **需解决的需求**: 根据三种角色 (管理员、人事、员工) 搭建友好的 Bootstrap 界面, 并处理好所有数据的异步收发与校验。
- **工作流程**:
 1. **静态切图**: 使用 HTML5/CSS3/Bootstrap 开发三套界面的 UI, 确保导航栏和表单清晰易用。
 2. **前端拦截与校验**: 在 JS 中实现非空/格式校验, 如果用户没填姓名, 直接在前端拦截, 不向后端发请求, 减轻服务器压力。
 3. **AJAX 联调 Mock**: 根据 D 的 API 文档, 编写 AJAX 请求逻辑。在 A 还没写好接口时, 先用本地写死的假 JSON 数据测试页面渲染逻辑。
 4. **状态反馈处理**: 处理后端返回的响应码 (如操作成功、Token 失效), 通过弹窗或红色提示语向用户进行友好反馈。

阶段三: 全链路联调与黑盒测试

当 A、B、C 都完成了各自的代码, 系统将进行组装。

D 的工作流程 (兼任 QA 测试):

1. **接口对齐**: 协助 A 和 C 进行跨域联调, 确保 AJAX 能够成功访问 C++ 后端, 且 JSON 解析不报错。
2. **越权黑盒测试**: D 登录“普通员工”账号, 尝试在浏览器控制台伪造 AJAX 请求调用“删除员工”接口, 验证 A 的 `AuthManager` 是否成功将其拦截。
3. **模糊查询验收**: 在前端输入关键词, 验证 B 的 `LIKE` 语句是否正确返回了多条记录。
4. **编译移植验证**: 要求 A 提交 CMake 源码, D 在 Windows 的 Visual Studio 或 MinGW 环境下尝试拉取代码并编译成 .exe, 验证跨平台能力。

阶段四: 交付物定稿

1. **D 完善全英文手册**: 将前期的设计图、API 文档以及测试结果填入模板, 完成“系统开发与实现 (10分)”部分。
2. **C 录制视频**: C 对自己写的 UI 最熟悉, 负责操作界面, 录制不超过 3 分钟的演示视频 (必须展示三种角色切换和模糊搜索)。
3. **B 导出数据**: B 负责从 openGauss 导出带有测试数据的 .sql 文件。
4. **A 统筹打包**: A 将源码、头文件、.exe (连同依赖的 dll 库)、手册、视频统一压缩为 学生姓名+企业人力管理系统.zip 进行提交。